Toegepaste Informatica

UCLL HOGESCHOOL

Full-Stack Software Development

Database Access with Java and Spring

R. Naudts, J. Pieck, J. Rombouts, B. Van Impe

CONTENTS

Java - Database

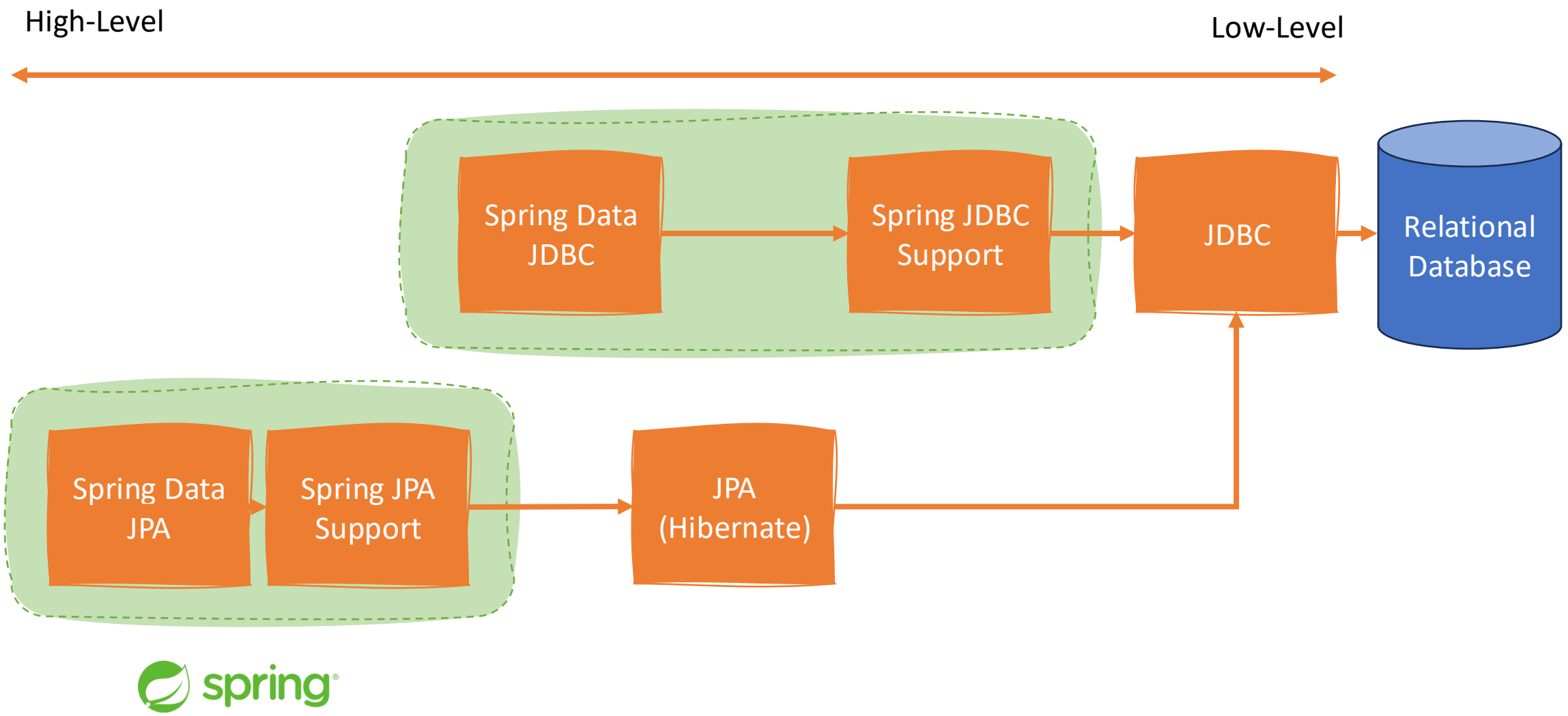
JAVA — DATABASE

- How can you connect with a database from Java?

- **J**ava **D**ata**B**ase **C**onnectivity (JDBC)
  - Low-level API: manage connection yourself and write queries manually

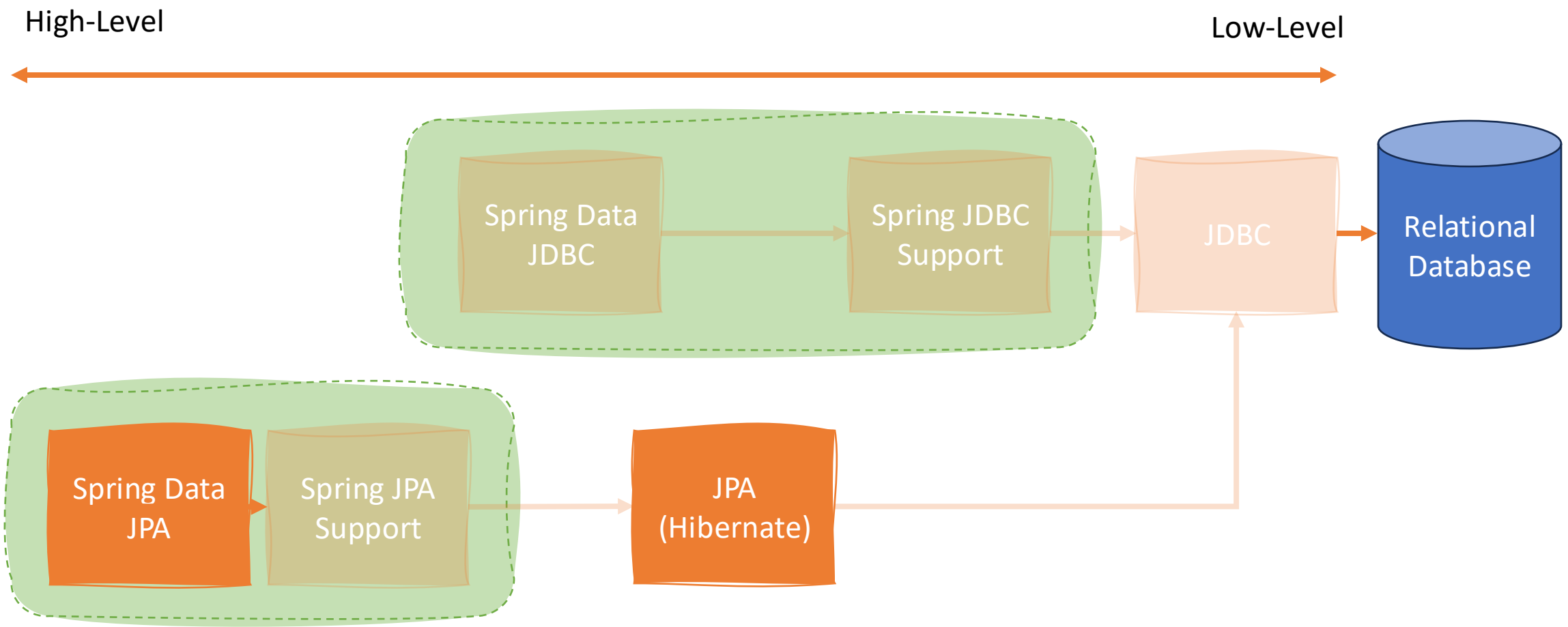
- **J**ava **P**ersistence **A**pi (JPA)
  - High-level API: developer performs operations on objects. The JPA implementation translates these operations to queries behind the scenes.
  - Multiple implementations, we use Hibernate as implementation.

JAVA — DATABASE



spring

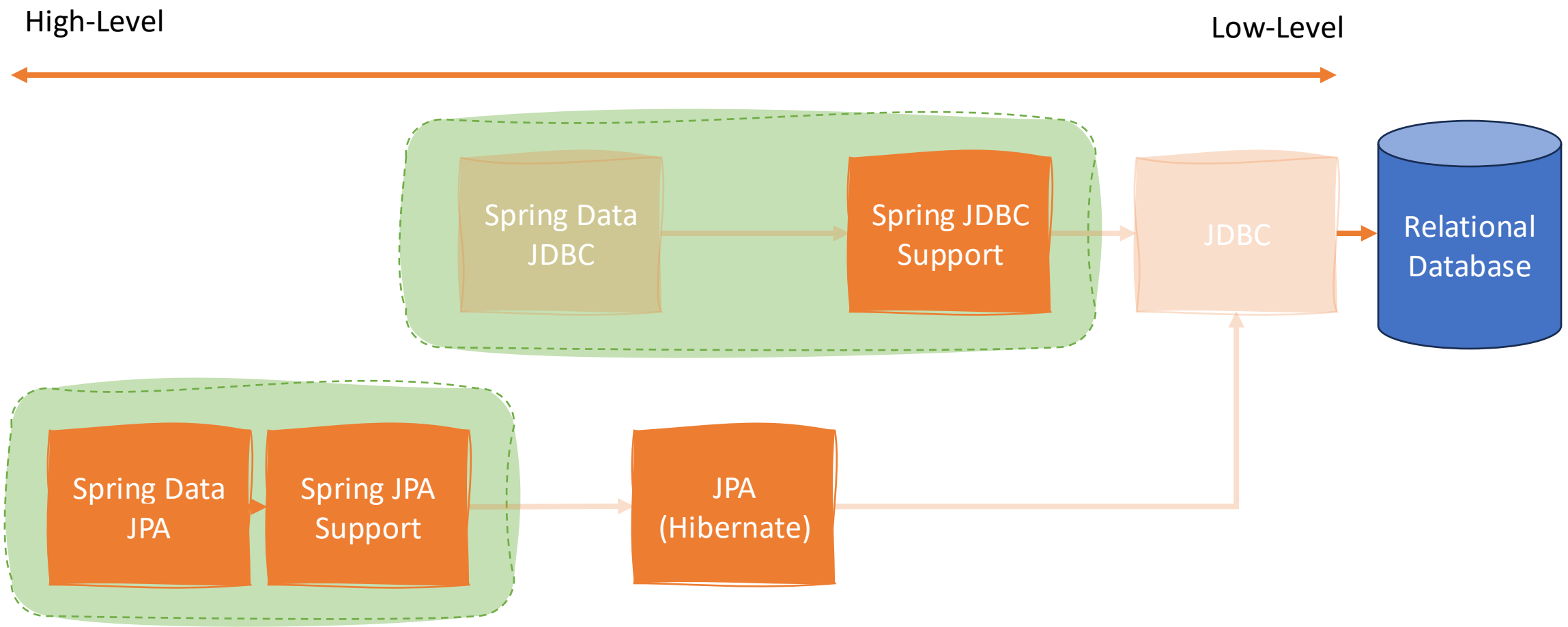
JAVA — DATABASE



spring

Introduced in Back-End Development

JAVA — DATABASE



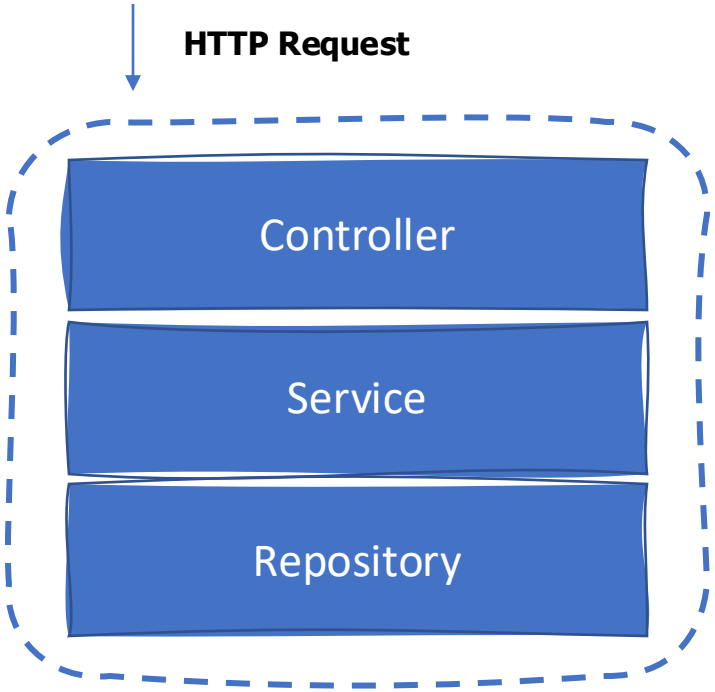
Focus of Full-Stack Software Development

JAVA — DATABASE

- Why only focus on the Spring enhanced versions?
  - Simplified code
    - For example, plain JDBC is (even more) verbose and error-prone
  - Easier exception handling
    - Exceptions are the same, regardless of the used technology
  - Unified transaction management
    - Transactions work with JDBC and JPA
  - Enhanced security
    - Eg, Spring JDBC disallows sending plain-text SQL to ensure protection against SQL injection
  - …

JDBC

WRITING A REPOSITORY WITH JDBC



Backend

Translates HTTP request to Java and translates Java back to a HTTP response

Guards application rules and gives commands to repositories to store, retrieve data

Communicates with the database and provides an API to use the database from within Java code

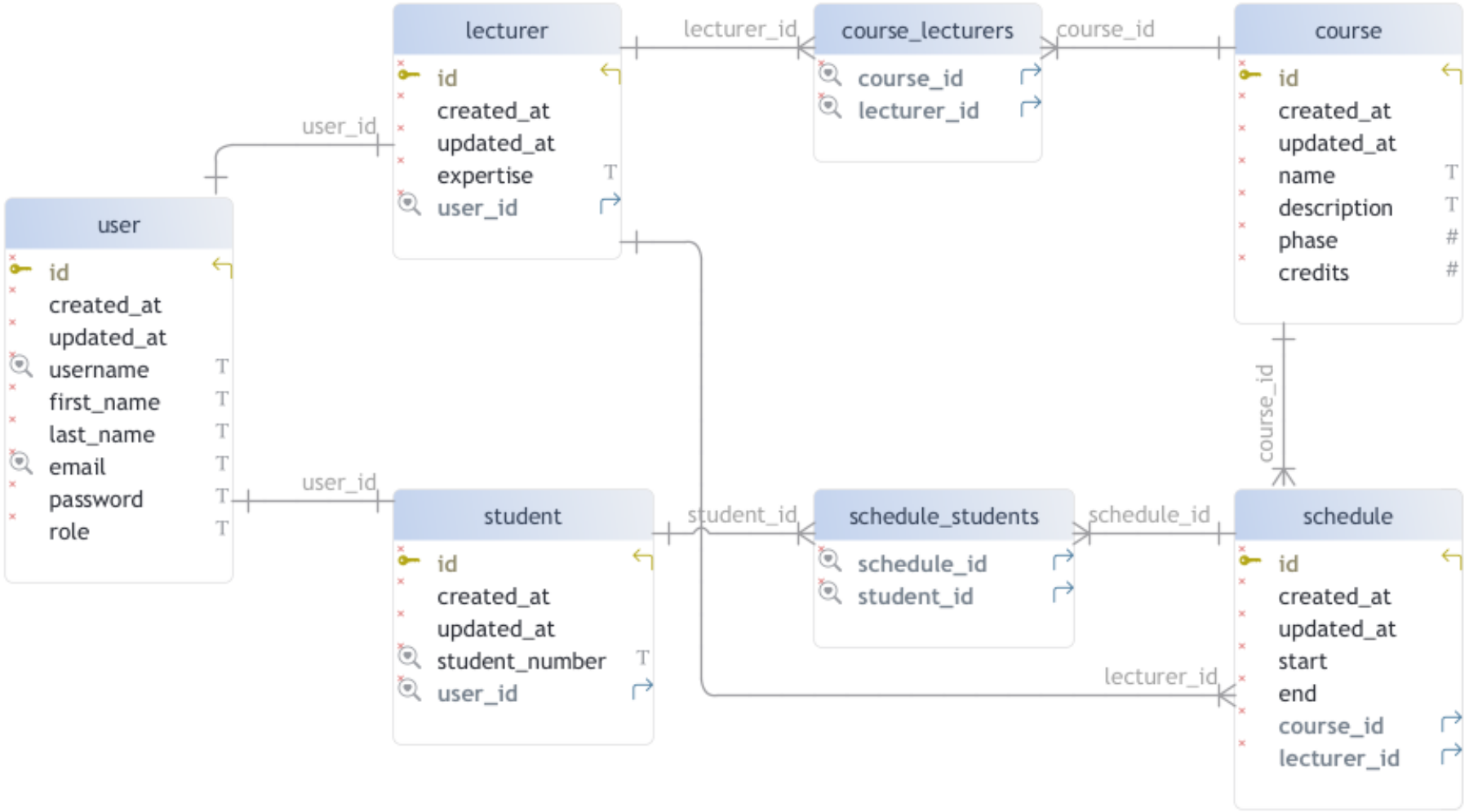
- The repository contains the interaction with the database
- Rewriting the repository to work with a database will not impact other classes since they rely on the repository for their database operations (the repository *encapsulates* the database access)

WRITING A REPOSITORY WITH JDBC

- Instead of defining a Spring Data JPA repository interface, we provide our own repository class
- We use Spring's 'JdbcClient' in this repository to interact with the database *directly*

- Caveats
  - Our classes do not need to be JPA entities!
  - Relationships are *hard,* since JPA is not tracking them:
    - You are responsible for keeping the foreign keys in the DB correct when updating
    - You are responsible for filling in the object-network when querying
      - Can be tricky if your repository method is used in lots of different ways

WRITING A REPOSITORY WITH JDBC



WRITING A REPOSITORY WITH JDBC

```
public class Course {

    // id is generated by DB when inserting
    private Long id;
    private Instant createdAt;
    private Instant updatedAt;
    private String name;
    private String description;
    private int phase;
    private int credits;

    // link with lecturers is through a join-table course_lecturers
    // with columns course_id and lecturer_id
    private List<Lecturer> lecturers;

    // ...
}
```

- We will focus on building a CRUD JDBC repository for the ‘Course’ class
- The ‘Course’ class corresponds with the ‘course’ and ‘course_lecturers’ tables

WRITING A REPOSITORY WITH JDBC

```
@Repository
public class JdbcCourseRepository {

    private final JdbcClient jdbcClient;

    public JdbcCourseRepository(JdbcClient jdbcClient) {
        this.jdbcClient = jdbcClient;
    }

    // ...
}
```

- Spring offers support for JDBC via the 'JdbcClient' class

JDBC – DELETE

```
@Override
public void deleteAll() {
    jdbcClient
            .sql(" truncate table course cascade ")
            .update();
}
```

- Specify the query to be executed via 'sql'
- Execute queries that modify the state of the database via 'update'
- Use cascade or multiple queries to clean-up all related tables
  - The query above cleans up multiple tables at the same time!
  - You can also use separate queries

JDBC – INSERT

```
@Override
public Course save(Course course) {
    Instant creationInstant = Instant.now();
    KeyHolder keyHolder = new GeneratedKeyHolder();

    jdbcClient.sql("""
                insert into course (created_at, updated_at, name, description, phase, credits)
                values (:createdAt, :updatedAt, :name, :description, :phase, :credits) returning id""")
            .param("createdAt", Timestamp.from(creationInstant))
            .param("updatedAt", Timestamp.from(creationInstant))
            .param("name", course.getName())
            .param("description", course.getDescription())
            .param("phase", course.getPhase())
            .param("credits", course.getCredits())
            .update(keyHolder);

    course.setId(keyHolder.getKeyAs(Long.class));

    for (Lecturer lecturer : course.getLecturers()) {
        jdbcClient.sql("""
                    insert into course_lecturers (course_id, lecturer_id) \
                    values (:courseId, :lecturerId)""")
                .param("courseId", course.getId())
                .param("lecturerId", lecturer.getId())
                .update();
    }

    return course;
}
```

- Specify the query via ‘sql’
- Specify values via ‘param’
- Execute via ‘update’

JDBC – INSERT

```
@Override
public Course save(Course course) {
    Instant creationInstant = Instant.now();
    KeyHolder keyHolder = new GeneratedKeyHolder();

    jdbcClient.sql("""
                insert into course (created_at, updated_at, name, description, phase, credits)
                values (:createdAt, :updatedAt, :name, :description, :phase, :credits) returning id""")
            .param("createdAt", Timestamp.from(creationInstant))
            .param("updatedAt", Timestamp.from(creationInstant))
            .param("name", course.getName())
            .param("description", course.getDescription())
            .param("phase", course.getPhase())
            .param("credits", course.getCredits())
            .update(keyHolder);

    course.setId(keyHolder.getKeyAs(Long.class));

    for (Lecturer lecturer : course.getLecturers()) {
        jdbcClient.sql("""
                    insert into course_lecturers (course_id, lecturer_id) \
                    values (:courseId, :lecturerId)""")
                .param("courseId", course.getId())
                .param("lecturerId", lecturer.getId())
                .update();
    }

    return course;
}
```

- Also insert data in related tables for the relationship with lecturers

JDBC – INSERT

```
@Override
public Course save(Course course) {
    Instant creationInstant = Instant.now();
    KeyHolder keyHolder = new GeneratedKeyHolder();

    jdbcClient.sql("""
                insert into course (created_at, updated_at, name, description, phase, credits)
                values (:createdAt, :updatedAt, :name, :description, :phase, :credits) returning id""")
            .param("createdAt", Timestamp.from(creationInstant))
            .param("updatedAt", Timestamp.from(creationInstant))
            .param("name", course.getName())
            .param("description", course.getDescription())
            .param("phase", course.getPhase())
            .param("credits", course.getCredits())
            .update(keyHolder);

    course.setId(keyHolder.getKeyAs(Long.class));

    for (Lecturer lecturer : course.getLecturers()) {
        jdbcClient.sql("""
                    insert into course_lecturers (course_id, lecturer_id) \
                    values (:courseId, :lecturerId)""")
                .param("courseId", course.getId())
                .param("lecturerId", lecturer.getId())
                .update();
    }

    return course;
}
```

- We can use a ‘KeyHolder’ to get access to the generated primary key
- Our query only returns the id instead of all fields

JDBC – FIND ALL

```
@Override
public List<Course> findAll() {
    return jdbcClient.sql("""
                select id, name as course_name, description, phase, credits
                from course""")
            .query(Course.class)
            .list();
}
```

- Specify the query via ’sql’
- Execute the query via ’query’
- Indicate that you expect multiple results via ‘list’

JDBC – FIND ALL

```
@Override
public List<Course> findAll() {
    return jdbcClient.sql("""
                select id, name as course_name, description, phase, credits
                from course""")
            .query(Course.class)
            .list();
}
```

- In the 'Query' method, you can specify the shape of the rows that you expect
- We specify that our rows should look like the 'Course' class
- This default mapping from row to object will only succeed for simple fields with matching names, all other fields will be left blank
  - The 'lecturers' field will not be filled-in!

JDBC – FIND ALL

```
@Override
public List<Course> findAll() {
    return jdbcClient.sql("""
                select id as course_id, name as course_name, description, phase, credits
                from course""")
            .query(new CourseRowMapper())
            .list();
}
```

- For non-trivial cases, you can specify a ‘RowMapper’ to do the mapping yourself
- A ‘RowMapper’ always maps **one row** to **one object**

JDBC – FIND ALL

```
public class CourseRowMapper implements RowMapper<Course> {

    @Override
    public Course mapRow(ResultSet rs, int rowNum) throws SQLException {
        Course course = new Course(
                rs.getString("course_name"),
                rs.getString("description"),
                rs.getInt("phase"),
                rs.getInt("credits")
        );
        course.setId(rs.getLong("course_id"));

        return course;
    }
}
```

- A ‘RowMapper’ always maps one row to one object
- !! With this ‘RowMapper’, the ‘lecturers’ field in our ‘Course’ class will remain empty !!

JDBC – FIND BY ID

```
@Override
public Optional<Course> findById(Long id) {
    return jdbcClient.sql("""
                select id as course_id, name as course_name, description, phase, credits
                from course
                where id = :id""")
            .param("id", id)
            .query(new CourseRowMapper())
            .optional();
}
```

- Specify the query via ’sql’
- Execute the query via ’query’
- Indicate that you expect one or no result via ‘optional’
- Mapping from row to object can be done via a result class or a ‘RowMapper’

JDBC – MAPPING RELATIONSHIPS

- A 'RowMapper' maps one row to one object
- Let's look at the following class:

```
public class Lecturer {

    private Long id;
    private Instant createdAt;
    private Instant updatedAt;
    private String expertise;

    private User user;
    private List<Course> courses;

    // ...
}
```

- What if we want to return it with **all fields** filled-in?

JDBC – MAPPING RELATIONSHIPS

```
public Optional<Lecturer> findById(Long id) {
    return jdbcClient.sql("""
                select lecturer.id as lecturer_id, expertise, public.user.id as user_id, username, first_name,
                    last_name, email, password, role
                from lecturer
                  inner join public.user on lecturer.user_id = public.user.id
                where lecturer.id = :id""")
            .param("id", id)
            .query(new LecturerRowMapper())
            .optional();
}
```

	lecturer_id bigint	user_id bigint	username text	first_name text	last_name text	email text	role text
1	1	2	johanp	Johan	Pieck	johan.pieck@ucll.be	lecturer

- If the relationship is to a single-value, we can include it in the query and map it via the 'RowMapper'

JDBC – MAPPING RELATIONSHIPS

```
public class LecturerRowMapper implements RowMapper<Lecturer> {

    @Override
    public Lecturer mapRow(ResultSet rs, int rowNum) throws SQLException {
        Lecturer lecturer = new Lecturer(
                rs.getString("expertise"),
                new UserRowMapper().mapRow(rs, rowNum)
        );
        lecturer.setId(rs.getLong("lecturer_id"));

        return lecturer;
    }
}
```

```
public class UserRowMapper implements RowMapper<User> {

    @Override
    public User mapRow(ResultSet rs, int rowNum) throws SQLException {
        User user = new User(
                rs.getString("username"),
                rs.getString("first_name"),
                rs.getString("last_name"),
                rs.getString("email"),
                rs.getString("password"),
                Role.valueOf(rs.getString("role").toUpperCase(Locale.ROOT))
        );
        user.setId(rs.getLong("user_id"));

        return user;
    }
}
```

- If the relationship is to a single-value, we can include it in the query and map it via the ‘RowMapper’

JDBC – MAPPING RELATIONSHIPS

- But what if the relationship is to multiple values?
  - Our 'RowMapper' only maps one row to one object

- Including the courses in the query leads to a more complex query result:

	lecturer_id bigint	user_id bigint	username text	first_name text	last_name text	email text	role text	course_id bigint	course_name text	description text	phase integer	credits integer
1	1	2	johanp	Johan	Pieck	johan.pieck@ucll.be	lecturer	5	Front-End Development	Learn how to build a front-end web application.	1	6
2	1	2	johanp	Johan	Pieck	johan.pieck@ucll.be	lecturer	6	Full-stack development	Learn how to build a full stack web application.	2	6

- The lecturer with id 1 has links to two courses

JDBC – MAPPING RELATIONSHIPS

```
public Optional<Lecturer> findById(Long id) {
    List<Lecturer> lecturers = jdbcClient.sql("""
                select lecturer.id as lecturer_id, expertise, public.user.id as user_id, username, first_name,
                    last_name, email, password, role, course.id as course_id, course.name as course_name, description, phase, credits
                from lecturer
                  inner join public.user on lecturer.user_id = public.user.id
                  inner join course_lecturers on lecturer.id = course_lecturers.lecturer_id
                  inner join course on course_lecturers.course_id = course.id
                where lecturer.id = :id""")
            .param("id", id)
            .query(new LecturerResultSetExtractor());

    return lecturers.isEmpty() ? Optional.empty() : Optional.of(lecturers.get(0));
}
```

- If the relationship is multi-valued, we can map the entire result-set in one go with a ‘ResultSetExtractor’
- The result is specified by the return value of the ResultSetExtractor, in this case a list

JDBC – MAPPING RELATIONSHIPS

```
public class LecturerResultSetExtractor implements ResultSetExtractor<List<Lecturer>> {

    // This method assumes that all rows for one lecturer come right behind each other! Enforce in query by ordering the resultset
    @Override
    public List<Lecturer> extractData(ResultSet rs) throws SQLException, DataAccessException {
        Lecturer currentLecturer = null;
        List<Long> lecturerIds = new ArrayList<>();
        List<Lecturer> lecturers = new ArrayList<>();

            while (rs.next()) {
            Long currentLecturerId = rs.getLong("lecturer_id");
            if (!lecturerIds.contains(currentLecturerId)) {
                Lecturer lecturer = new LecturerRowMapper().mapRow(rs, rs.getRow());
                lecturerIds.add(currentLecturerId);
                lecturers.add(lecturer);
                currentLecturer = lecturer;
            }
            currentLecturer.addCourse(new CourseRowMapper().mapRow(rs, rs.getRow()));
        }

        return lecturers;
    }
}
```

- If the relationship is multi-valued, we can map the entire result-set in one go with a ‘ResultSetExtractor’

JDBC – MAPPING RELATIONSHIPS

- Writing a good ResultSetExtractor is *hard*
  - Its implementation is deeply coupled with all details from the result-set
  - The implementation from the previous slide would not work with a different row-order
- The complexity only increases with the number of relationships that you want to include
  - A course also references lecturers…
  - Lecturers also reference courses…
- Where do you stop? What do you include?
  - You need to know what your code uses and account for that…
  - … or introduce hard-boundaries in your domain model

JDBC – CONCLUSIONS

- You can write any query with the discussed techniques
- But…
  - Writing JDBC repositories correctly is very hard
  - And very boring: the structure of every repository is always the same

- JPA addresses those negatives, but takes away control in the process due to the higher abstraction
- Another solution is Spring Data JDBC (not covered) with simplifies JDBC usage without the complexity of JPA but **requires** the hard-boundaries introduced by *aggregates* (see next class)

JPA

JPA – ORM

- The mapping of Java objects to database tables, and vice-versa is called **O**bject-**R**elational **M**apping (ORM)
  - An ORM library works as
    - a **bridge** between

      a **relational database (tables and records)**

      and

      a **Java application (classes and objects)**

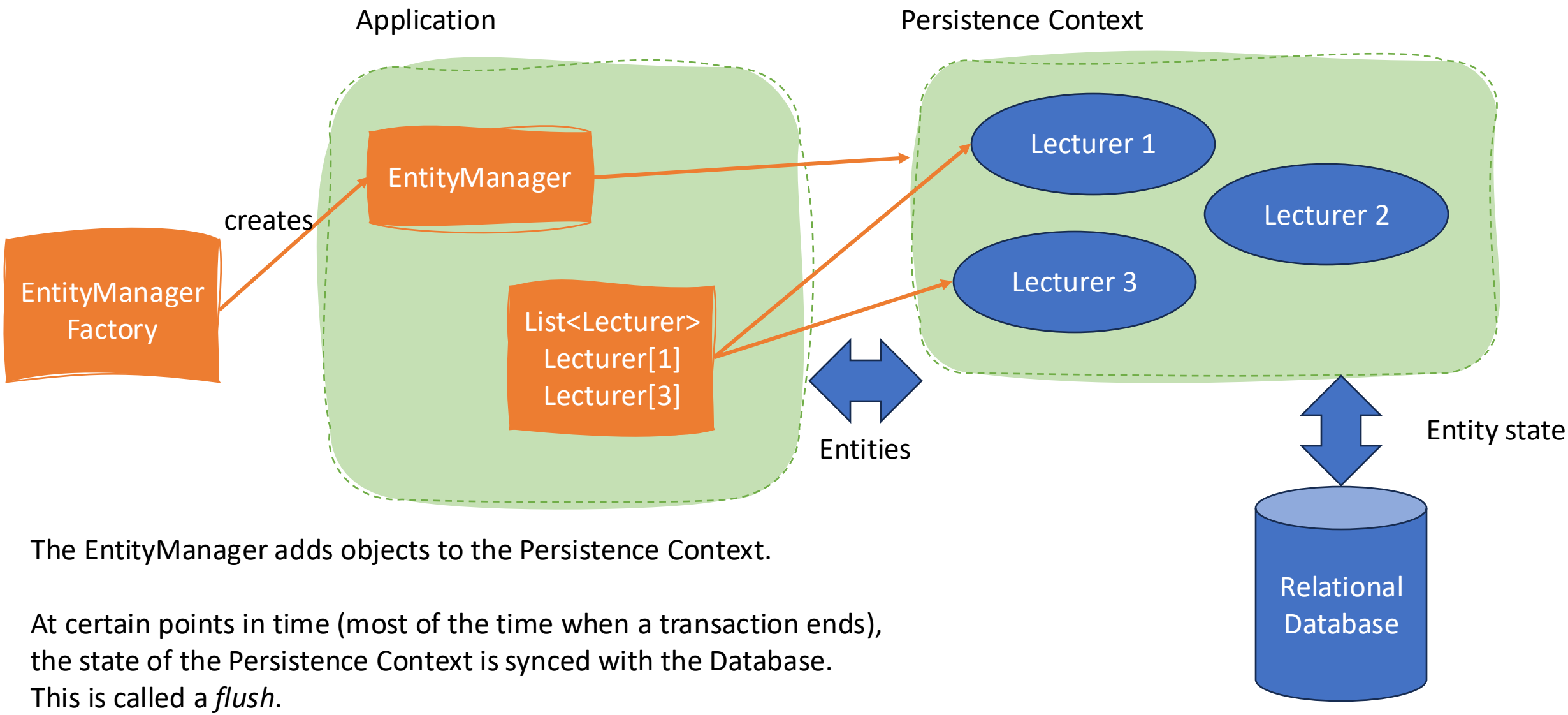
- The purpose of an ORM library is to automatically sync operations done on Java objects to the database
  - For example, updating fields of an object leads to an automatic update query behind the scenes

JPA – IMPORTANT CONCEPTS

- Entity: an object whose state can be stored to and retrieved from the database through JPA

- EntityManager: manages a unit of work and persistent objects therein: the *persistence context*
- EntityManagerFactory: responsible for creating an EntityManager for every request
- Persistence Context: all entities that are currently being managed by JPA. By default, a persistence context is opened and closed for every request.

JPA – IMPORTANT CONCEPTS



The EntityManager adds objects to the Persistence Context.

At certain points in time (most of the time when a transaction ends), the state of the Persistence Context is synced with the Database. This is called a *flush*.

ENTITYMANAGER API

Method	Description
persist(Object o)	Adds new entity to the Persistence Context SQL: insert into table…
remove(Object o)	Removes entity from the Persistence Context SQL: delete from table…
find(Class entity, Object pk)	Find by primary key SQL: select * from table where id = ?
createQuery(String jpqlString)	Create a JPQL query
flush()	Force changed entity state to be written to database immediately

- Why is there no 'update' method?
  - Saving changes to an entity in the persistence context happens automatically when the persistence context is synced to database at the end of a transaction
  - So 'find' is enough to update the values of an entity
  - See next class for more details on this

WRITING A REPOSITORY WITH JPA

```
@Repository
public class JpaCourseRepository {

    private final EntityManager entityManager;

    public JpaCourseRepository(EntityManager entityManager) {
        this.entityManager = entityManager;
    }

    // ...
}
```

- Let Spring autowire the 'EntityManager' to work with JPA directly

JPA – INSERT

```
@Override
public Course save(Course course) {
    entityManager.persist(course);
    return course;
}
```

- Adds object to the EntityManager
- When a flush happens, the object will be saved in the database

JPA – FIND ALL

```
@Override
public List<Course> findAll() {
    return entityManager.createQuery("select c from Course c", Course.class).getResultList();
}
```

- Use ‘createQuery’ to specify the query you want to execute
- Use ‘getResultList’ to indicate that you expect multiple results

JPA – JPQL

- Queries in JPA are written in JPQL
- The most important thing to remember is that you query your entities, not your database model!
  - It is the job of JPA to translate queries directed to entities to your database model

- The full JPQL specification can be found here
  - If you can write SQL, then it is not hard to write JPQL
  - The examples in the slide should suffice for almost all your needs

JPA – FIND BY ID

```
@Override
public Optional<Course> findById(Long id) {
    return Optional.ofNullable(entityManager.find(Course.class, id));
}
```

- Use ‘find’ to search by PK

JPA – DELETE

```
@Override
public void deleteAll() {
    entityManager.createQuery("delete from Course").executeUpdate();
}
```

- You delete the entity
- JPA will figure out which tables need to be deleted

JPA – CUSTOM QUERY

```
@Override
public List<Schedule> findByLecturer_User_Username(String
username) {
    return entityManager
            .createQuery("""
                select s from Schedule s
                  join s.lecturer l
                  join l.user u
                where u.username = :username""", Schedule.class)
            .setParameter("username", username)
            .getResultList();
}
```

- Using JPQL it is possible to write any query you want
- Since you query entities, the queries are generally simpler than their SQL equivalent

JPA – CONCLUSIONS

- Seeing the JDBC equivalent helps to appreciate the help that JPA offers
- A lot of complexity is abstracted away
  - Relationship references are handled through entity annotations
  - Querying entities leads to simpler queries
    - Eg, for many-to-many, the join-table can be skipped
- Writing all of your JPA repositories by hand is still quite boring, since they are all very similar…

SPRING DATA JPA

- This brings us back to Spring Data JPA, an abstraction on top of JPA, provided by Spring
- Looking back at the structure of JPA repositories, we can see how Spring is able to generate the implementation by simply looking at the method names

SPRING DATA JPA

```
public interface SpringDataJpaScheduleRepository extends JpaRepository<Schedule, Long> {

    List<Schedule> findByLecturer_User_Username(String username);

    boolean existsByCourse_IdAndLecturer_Id(Long courseId, Long lecturerId);
}
```

- Save, delete, findOne and findAll have very similar implementations every time. Spring can generate them by knowing the type of the Entity that the Repository should operate on.
- Custom queries can be translated to a JPQL query when the method name is well structured

SPRING DATA JPA – ONE-OFF CUSTOM QUERIES

```
@Query("""
    select s from Schedule s
    where s.course.id = :courseId
      and s.lecturer.id = :lecturerId""")
boolean existsByCourse_IdAndLecturer_Id(Long courseId, Long lecturerId);
```

- You can combine Spring Data JPA with custom queries by using the @Query annotation
  - This is useful for readability, convenience or really complex queries
- The method-name can be chosen freely if @Query is present
- By default, the expected query is a JPQL query

SPRING DATA JPA – ONE-OFF CUSTOM QUERIES

```
@Query(value = """
    select id as user_id
    from public.user
    where username = :username""", nativeQuery = true)
Optional<Long> findIdByUsername(String username);
```

- You can run a native SQL query by changing the 'nativeQuery' flag
  - Use with care!

SPRING DATA JPA – CONCLUSIONS

- In general, using JPA over JDBC is recommended unless the low-level control is critical for performance or other technical reasons
- When working with JPA, there is no reason to use plain JPA over Spring Data JPA
  - But knowing plain JPA helps with understanding Spring Data JPA
- Add custom queries to your Spring Data repositories to cover the complex cases
  - For even more complex cases, it is sometimes useful to add *some* custom JDBC code to a codebase